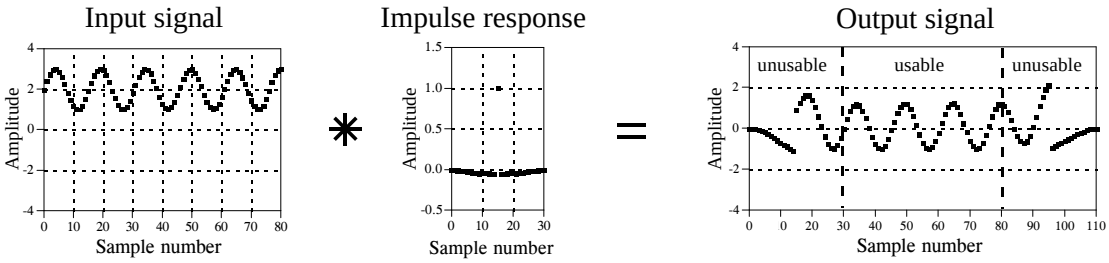

FIGURE 6-10
End effects in convolution. When an input signal is convolved with an *M* point impulse response, the first and last *M*-1 points in the output signal may not be usable. In this example, the impulse response is a high-pass filter used to remove the DC component from the input signal.

sample in the impulse response. Line 230 provides the multiplication of each sample in the impulse response, H[J%], with the appropriate sample from the input signal, X[I%-J%], and adds the result to the accumulator.

In line 230, the sample taken from the input signal is: X[I%-J%]. Lines 210 and 220 prevent this from being outside the defined array, X[0] to X[80]. In other words, this program handles undefined samples in the input signal by *ignoring* them. Another alternative would be to define the input signal's array from X[-30] to X[110], allowing 30 zeros to be padded on each side of the true data. As a third alternative, the FOR-NEXT loop in line 180 could be changed to run from 30 to 80, rather than 0 to 110. That is, the program would only calculate the samples in the output signal where the impulse response is *fully immersed* in the input signal. The important thing is that you must use one of these three techniques. If you don't, the program will crash when it tries to read the out-of-bounds data.

```
100  'CONVOLUTION USING THE OUTPUT SIDE ALGORITHM
110                              '
120 DIM X[80]                    'The input signal, 81 points
130 DIM H[30]                    'The impulse response, 31 points
140 DIM Y[110]                   'The output signal, 111 points
150                              '
160 GOSUB XXXX                   'Mythical subroutine to load X[ ] and H[ ]
170                              '
180 FOR I% = 0 TO 110            'Loop for each point in Y[ ]
190   Y[I%] = 0                  'Zero the sample in the output array
200   FOR J% = 0 TO 30           'Loop for each point in H[ ]
210     IF (I%-J% < 0)           THEN GOTO 240
220     IF (I%-J% > 80)          THEN GOTO 240
230     Y(I%) = Y(I%) + H(J%) * X(I%-J%)
240   NEXT J%
250 NEXT I%
260                              '
270 GOSUB XXXX                   'Mythical subroutine to store Y[ ]
280                              '
290 END
```

TABLE 6-2